THE YENEPOYA INSTITUTE OF ARTS, SCIENCE,
COMMERCE AND MANAGEMENT
Project- High Level Design

On

Hardened Cloud Infrastructure for Agricultural Data

Student Name:
Azzam Haris NK
23BCCE102

Table of Contents

1. Introduction

This document presents the High-Level Design (HLD) of the Hardened Cloud Infrastructure for Agricultural Data system. The system is designed to accept sensitive agriculture data files through a web portal, encrypt every file using AES-256-CBC before writing it to disk, store encrypted blobs in a secure vault, and allow authorised retrieval through password-based decryption. It is inspired by the principle of enterprise-grade Data-at-Rest protection used by real financial and agricultural data platforms.

The project simulates a Security Operations workflow for agricultural organisations — from the moment a farmer uploads a soil report or yield projection to the moment the file is retrieved securely — making it suitable for academic study as well as real-world application in the domain of cloud security and cryptography.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and component interactions
- Application design including encryption process flow and information flow
- Data design including the .agv encrypted file format, metadata schema, access mechanism and retention policies
- Non-functional requirements such as security and performance
- Interface design for the Flask-based web portal

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the Low-Level Design (LLD) document.

1.2. Intended Audience

This document is intended for the following stakeholders:

- Academic evaluators and faculty reviewing the project submission
- Developers or students who may extend or build upon this system
- Cloud security professionals seeking to understand encryption-first storage design
- Anyone interested in the application of Python cryptography in agriculture data protection

1.3. System Overview

The Hardened Cloud Infrastructure for Agricultural Data is a real-time secure file storage system built using Python, the PyCA cryptography library, Flask and HTML/CSS/JavaScript. It accepts any agriculture data file, encrypts it with AES-256-CBC using a PBKDF2-derived key, stores the ciphertext in a custom .agv format, and presents all vault operations through an interactive web-based portal.

The system mimics the workflow of a real cloud storage security environment:

- Upload: User selects a file and provides an encryption password through the web portal
- Derive Key: PBKDF2-HMAC-SHA256 with 310,000 iterations and a random 256-bit salt produces the AES key
- Encrypt: AES-256-CBC with a random 16-byte IV encrypts the file; PKCS7 padding applied
- Integrity: HMAC-SHA256 computed over IV + ciphertext (Encrypt-then-MAC pattern)
- Store: Encrypted blob saved as .agv file; metadata logged to vault_metadata.json
- Retrieve: HMAC verified before decryption; original file returned to browser on password match

The system is built using the following technology stack:

Component	Technology	Purpose

File Encryption	Python cryptography (PyCA)	AES-256-CBC encrypt/decrypt with PKCS7 padding
Key Derivation	PBKDF2-HMAC-SHA256	Derives 256-bit key from password + 32-byte salt
Integrity Check	HMAC-SHA256	Encrypt-then-MAC tamper detection on every file
Web Server	Flask + REST API	Upload, list, download, delete and stats endpoints
Web Portal	HTML + CSS + JS	Drag-and-drop upload, vault dashboard, decrypt modal
Metadata Store	JSON flat file	Stores file ID, name, size, algorithm, timestamp

1.4. Assumptions

- Python 3.x environment is configured on the host system
- Required libraries (Flask, cryptography, Flask-CORS) are installed
- Application runs on localhost or internal network during demonstration
- User has permission to create directories and write files on the host
- Log and vault directories are writable for persistent record keeping

2. System Design

2.1. Application Design

The system is built as two cooperating Python applications running side by side. The core encryption engine (crypto_engine.py) is a pure Python module that handles all AES-256-CBC operations, PBKDF2 key derivation, HMAC integrity, and the custom .agv binary format. The web application (app.py) is a Flask server that exposes REST API endpoints and hosts the HTML portal. Both components share state only through the vault_metadata.json file and the encrypted_vault/ directory, keeping them fully decoupled.

Component Interaction — UML Component Diagram

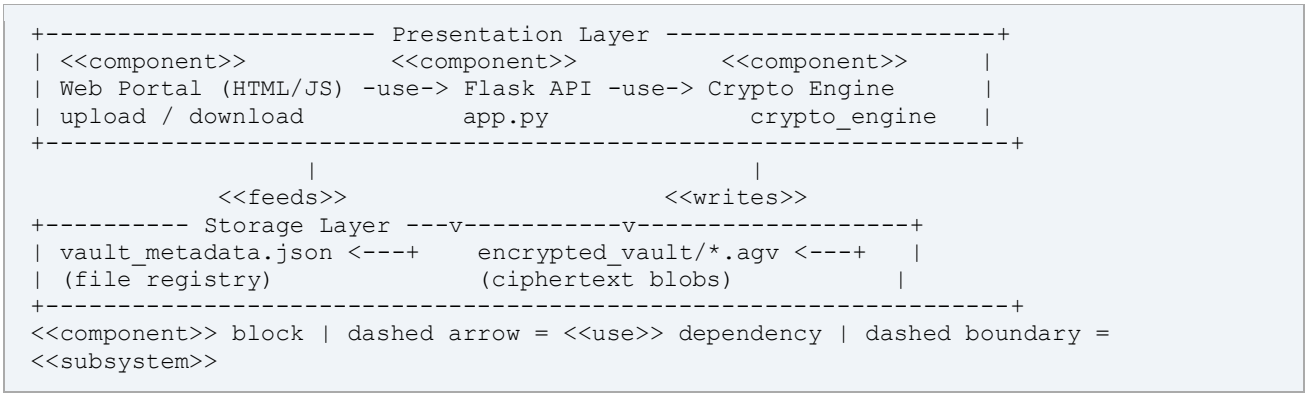

Figure 1.1

2.2. Process Flow

The process flow of the system follows a sequential encryption pipeline:

- Step 1 - Initialization: Flask server starts, vault and upload directories created/verified on startup
- Step 2 - Upload: User selects file and enters password on the web portal; POST to /api/upload
- Step 3 - Key Derivation: Random 32-byte salt generated; PBKDF2-HMAC-SHA256 (310,000 iterations) produces 256-bit AES key
- Step 4 - Encryption: Random 16-byte IV generated; AES-256-CBC encrypts file content with PKCS7 padding
- Step 5 - Integrity: HMAC-SHA256 computed over IV + ciphertext (Encrypt-then-MAC)
- Step 6 - Persistence: .agv blob (MAGIC|SALT|IV|HMAC|TS|FNAME|CIPHER) written to encrypted_vault/
- Step 7 - Metadata: Entry with UUID, filename, sizes, algorithm and timestamp appended to vault_metadata.json
- Step 8 - Dashboard: Portal refreshes vault listing; stat cards updated in real time

Figure 1.2 — Encryption Process Flowchart

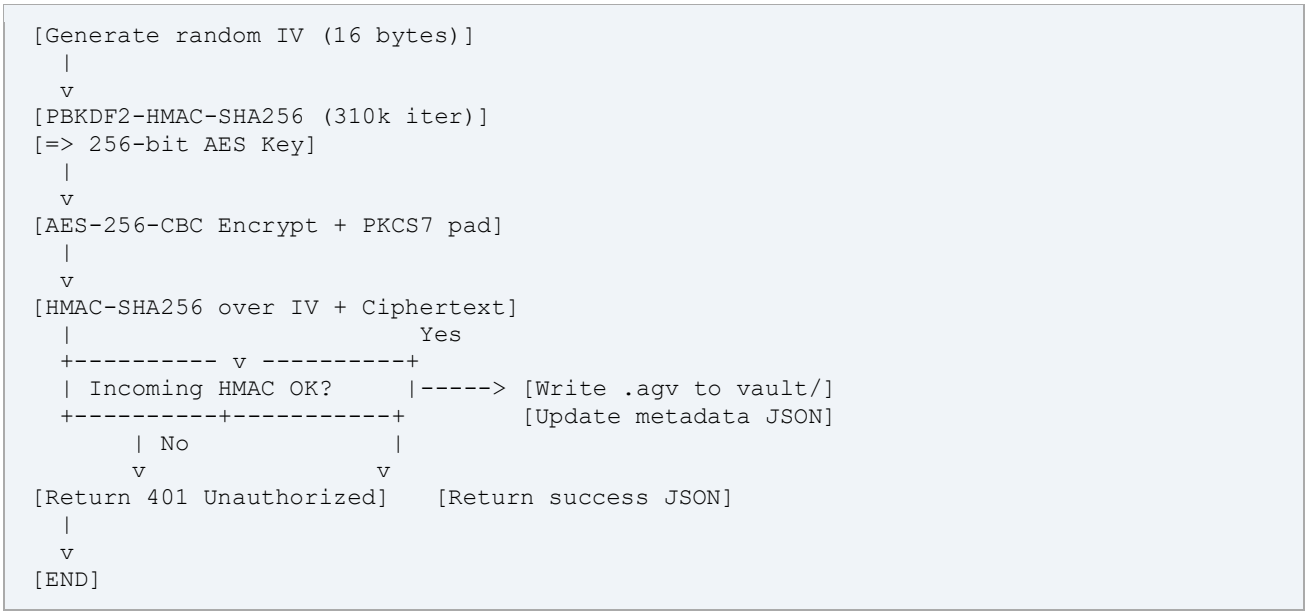

Figure 1.2

2.3. Information Flow

Data flows through the system in the following direction:

- Farmer/User → Selects file + enters password on web portal
- Browser → POST /api/upload with multipart file + password form field
- Flask server → Receives file, saves temporary plaintext to /uploads/
- Crypto engine → Generates SALT + IV; PBKDF2 derives AES key from password + SALT
- AES-256-CBC → Encrypts plaintext; HMAC-SHA256 seals the ciphertext
- Vault writer → Builds .agv binary blob; saves to encrypted_vault/<uuid>.agv
- Metadata logger → Appends {id, filename, size, algorithm, timestamp} to vault_metadata.json
- Flask /api/files → Reads metadata and returns file listing to browser
- Browser dashboard → Renders vault grid, stat cards and encryption pipeline in real time

2.4. Components Design

The system is composed of the following major components:

Component	Description
Crypto Engine	Implements AES-256-CBC encryption/decryption, PBKDF2-HMAC-SHA256 key derivation, HMAC-SHA256 integrity and .agv binary format read/write
Port Listener / Flask	Flask REST API server binding to port 5000; handles HTTP requests, file lifecycle management and routing to crypto engine
Key Derivation Unit	PBKDF2-HMAC-SHA256 with 310,000 iterations and a per-file 32-byte random salt; converts user passwords to 256-bit AES keys
HMAC Integrity Engine	Computes and verifies HMAC-SHA256 over IV+ciphertext (Encrypt-then-MAC); constant-time comparison prevents timing attacks
Vault Storage	Filesystem directory (encrypted_vault/) holding .agv encrypted blobs; metadata tracked in vault_metadata.json JSON flat file
Web Portal Dashboard	HTML/CSS/JS frontend providing drag-and-drop upload, encrypted vault grid, stats panels, decrypt modal and encryption pipeline diagram

Secure Delete Engine	Overwrites encrypted file bytes with os.urandom() before os.remove(); prevents file recovery from disk sectors

2.5. Key Design Considerations

- AES-256-CBC is used with a random 16-byte IV per encryption so that two encryptions of identical files always produce different ciphertext (semantic security)
- PBKDF2-HMAC-SHA256 with 310,000 iterations follows the OWASP 2023 minimum recommendation; makes brute-force attacks computationally infeasible
- Encrypt-then-MAC pattern ensures HMAC is computed after encryption and verified before decryption — prevents padding oracle attacks
- Constant-time HMAC byte comparison (_constant_time_compare) mitigates remote timing side-channel attacks
- Original filename is stored inside the encrypted .agv blob — not exposed in the filesystem filename — preserving filename privacy
- Vault and dashboard are decoupled — connected only via vault_metadata.json and the encrypted_vault/ directory

2.6. API Catalogue

The system exposes internal REST endpoints through the Flask web server. These are consumed by the portal JavaScript and the CLI tool.

Method	Endpoint	Response	Description
POST	`/api/upload`	JSON	Upload and encrypt file; body: multipart/form-data with file and password
GET	`/api/files`	JSON	Returns metadata array of all vault files without exposing ciphertext
POST	`/api/download/:id`	File stream	Decrypt and stream original file; body: JSON {password}; 401 on wrong password
DELETE	`/api/delete/:id`	JSON	Securely overwrite and delete encrypted file from vault
GET	`/api/stats`	JSON	Vault statistics: total files, total original bytes, total encrypted bytes
GET	`/api/health`	JSON	Server health check: {status, service, version}

3. Data Design

3.1. Data Model

The primary data model used by the system is a JSON array of vault file records stored in vault_metadata.json, combined with the binary .agv encrypted file format. Each metadata entry is a flat dictionary with the following schema:

Field	Data Type	Description
`id`	String (UUID4)	Unique file identifier used in all API endpoints
`original_filename`	String	Original filename of the uploaded agriculture data file
`original_size`	Integer	Size of the plaintext file in bytes before encryption
`encrypted_size`	Integer	Size of the AES ciphertext portion of the .agv file
`encrypted_path`	String	Absolute filesystem path to the .agv encrypted file
`algorithm`	String	'AES-256-CBC' — encryption algorithm identifier
`kdf`	String	'PBKDF2-HMAC-SHA256 (310000 iterations)' — KDF descriptor
`integrity`	String	'HMAC-SHA256 (Encrypt-then-MAC)' — integrity scheme
`uploaded_at`	Integer	Unix timestamp (seconds) of encryption time
`status`	String	'encrypted' — current lifecycle state

The following internal function interfaces are defined in crypto_engine.py:

Function	Input	Output	Description
`encrypt_file(path, pwd)`	str, str	dict	Encrypts file; returns metadata result dict
`decrypt_file(path, pwd, out)`	str, str, str	dict	Verifies HMAC; decrypts and saves original
`derive_key(password, salt)`	str, bytes	bytes	PBKDF2-HMAC-SHA256 → 32-byte AES key
`compute_hmac(key, data)`	bytes, bytes	bytes	Returns 32-byte HMAC-SHA256 over data
`get_file_metadata(path)`	str	dict	Reads .agv header without decrypting content
`_constant_time_compare(a, b)`	bytes, bytes	bool	Constant-time HMAC byte comparison

3.2. Data Access Mechanism

- Vault Metadata: vault_metadata.json — plain JSON array read and written on each upload or delete operation
- Thread Safety: threading.Lock() mutex (metadata_lock) prevents concurrent write corruption of the JSON file

- Encrypted Files: Flask endpoints retrieve the filepath from metadata and open the .agv file directly for decryption
- Dashboard Reads: GET /api/files reads the full JSON metadata and returns serialised file listing to the browser

3.3. Data Retention Policies

- Encrypted Vault: retained indefinitely on disk until explicitly deleted by the user through the dashboard or DELETE API
- Temporary Uploads: plaintext file in /uploads/ is deleted immediately after encryption completes; never persists
- Decrypted Temp Files: written to /decrypted_temp/ for download; should be cleared by a maintenance routine in production
- Metadata Records: entry removed atomically from vault_metadata.json when the corresponding .agv file is deleted

3.4. Data Migration

The system does not involve database migration — the data store is a flat JSON metadata file combined with binary .agv files. If the system is moved to a new host, copying the vault_metadata.json file and the encrypted_vault/ directory is sufficient to preserve the complete vault history. If required, metadata can be imported into PostgreSQL or MongoDB and .agv files can be stored in an object store such as AWS S3 or MinIO for production deployments, without modifying the crypto engine core.

4. Interfaces

The system provides a single web-based interface through the Flask portal, accessible at http://127.0.0.1:5000. The interface consists of the following elements:

UI Element	Type	Description
Encryption Pipeline	HTML divs + CSS	Visual five-step flow: Upload → PBKDF2 → AES-256-CBC → HMAC → Vault Storage
Stat Cards (x4)	HTML cards	Encrypted Files count, Original Data size, Encrypted Size, Algorithm label
Drop Zone	HTML div + JS	Drag-and-drop or click-to-select file upload area with file type detection
Password Fields	HTML inputs	Encryption password + Confirm password fields; client-side match validation
Algorithm Info Banner	HTML div	Displays AES-256-CBC details and PBKDF2 parameters to educate the user
Progress Bar	HTML + CSS	Animated upload/encryption progress with step labels
Vault Table	HTML table	Lists all encrypted files: filename, original size, encrypted size, algorithm, date, actions
Decrypt Modal	HTML overlay	Password input dialog triggered on Download; HMAC error displayed on wrong password
Toast Notifications	HTML div	Success/error/warning messages shown top-right on all operations

5. State and Session Management

The AgriVault system does not maintain user sessions as it has no user login. State is managed through the following components:

- Vault State: persistent across restarts because the encrypted_vault/ directory and vault_metadata.json live on disk
- API State: Flask is stateless — every HTTP request is self-contained; no server-side session variables are maintained
- Encryption State: passwords are never stored anywhere; they exist only in memory during the duration of one encrypt or decrypt operation
- Browser State: lightweight JavaScript variables track the selected file and current decrypt modal target; reset on page refresh
- Thread Safety: a threading.Lock() guards the vault_metadata.json write path to prevent race conditions on concurrent uploads

6. Caching

The system does not use an in-memory cache for vault metadata. Every GET /api/files and GET /api/stats request reads the vault_metadata.json file fresh from disk. This keeps the implementation simple and guarantees the dashboard always reflects the latest encrypted file state. AES key derivation (PBKDF2) results are intentionally never cached — re-deriving on each request is a security requirement to prevent key material from persisting in memory.

For production deployments at scale, a Redis caching layer could be added between vault_metadata.json and Flask to avoid repeated disk reads — this is intentionally out of scope for the academic version. Browser-side caching of the vault listing could also be implemented with ETag or Last-Modified headers if needed.

7. Non-Functional Requirements

7.1. Security Aspects

- AES-256-CBC encryption ensures all agriculture data files are unreadable at rest without the correct password
- PBKDF2-HMAC-SHA256 with 310,000 iterations prevents brute-force password recovery from the encrypted files
- Random 256-bit salt per file prevents rainbow table and precomputation attacks across the vault
- Encrypt-then-MAC (HMAC-SHA256) ensures tamper detection; HMAC verified before decryption begins
- Constant-time HMAC comparison mitigates remote timing side-channel attacks
- Temporary plaintext file deleted immediately after encryption — never persists on disk
- Secure deletion overwrites file bytes with os.urandom() before os.remove() to prevent disk recovery
- Original filenames stored inside the encrypted blob — not exposed in the vault filesystem

7.2. Performance Aspects

- AES-256-CBC throughput is approximately 10–50 MB/s on typical hardware using the PyCA OpenSSL backend
- PBKDF2 key derivation takes 0.3–0.8 seconds intentionally — this computational cost deters brute-force attacks

- Flask REST API response time for file listing is under 200 ms for vault sizes up to 1,000 entries
- Upload encryption completes in under 2 seconds for files up to 10 MB including network and HMAC computation
- Thread-level lock on metadata ensures correctness without significant throughput impact at academic scale

7.3. Limitations

- PBKDF2 derivation (0.3–0.8 seconds) may feel slow on upload — intentional security trade-off
- vault_metadata.json flat-file storage is not suitable for high-volume production vaults with thousands of files
- No multi-user access control — the system is a single-user academic demonstration
- Decrypted temporary files in /decrypted_temp/ are not auto-cleaned in the current version
- Flask development server is single-threaded by default — use Gunicorn for concurrent production use

7.4. Future Enhancements

- Deploy on cloud server (AWS S3 / Azure Blob) to store .agv files in a managed object store
- Add per-user authentication with JWT tokens to support multi-farmer vaults
- Migrate metadata from JSON to PostgreSQL for indexing and search on large vaults
- Add RSA asymmetric key wrapping so each farmer's AES key is sealed with their public key
- Integrate with Wazuh or Splunk for real SOC-level audit logging of vault access events

7.5. Testing Summary

- Encryption and decryption roundtrip tested for small, large and binary files including empty files
- Wrong password correctly returns HMAC verification failure (401) without decrypting any bytes
- Tampered ciphertext (bit-flip) correctly detected by HMAC before decryption attempt
- Two encryptions of the same file produced different ciphertext confirming IV and salt randomness
- Secure delete confirmed: file bytes overwritten with random data before removal from vault

8. References

- Python cryptography Library Documentation (PyCA) — https://cryptography.io/en/latest/
- NIST FIPS 197 — Advanced Encryption Standard (AES) — https://csrc.nist.gov/publications/fips/fips197/fips-197.pdf
- NIST SP 800-38A — Recommendation for Block Cipher Modes of Operation (CBC) — https://csrc.nist.gov/publications/detail/sp/800-38a/final
- OWASP Password Storage Cheat Sheet (2023) — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- RFC 8018 — PKCS #5 v2.1: Password-Based Key Derivation (PBKDF2) — https://www.rfc-editor.org/rfc/rfc8018
- RFC 2104 — HMAC: Keyed-Hashing for Message Authentication — https://www.rfc-editor.org/rfc/rfc2104
- Flask Official Documentation — https://flask.palletsprojects.com
- NIST SP 800-88 — Guidelines for Media Sanitisation (Secure Deletion) — https://csrc.nist.gov/publications/detail/sp/800-88/rev-1/final

YENEPOYA INSTITUTE OF ARTS, SCIENCE AND COMMERCE | PROJECT 2026 | End of Document